Protocole de comparaison des librairies d'analyse des trajectoires : TraMineR et TraMineRextras

Garance Malnoë

Avril 2026

Contents

1 Introduction **2**

2 Formats de données **2**

3 Données exemples proposées et simulations **4**

4 Données fil-rouge **4**
4.1 Extrait du Swiss Household Panel `extract_shp` . . . 5

5 Gestion des données **5**
5.1 Query . . . 6
5.2 Static . . . 6
5.3 Time . . . 6
5.4 Pattern . . . 7
5.5 Length . . . 7
5.6 Changement de granularité . . . 7

6 Descriptions des séquences et visualisations **7**
6.1 Alphabet . . . 7
6.2 Visualisation des séquences . . . 7
6.3 Fréquences des séquences . . . 9
6.4 Fréquences des états co-occurents . . . 9
6.5 Séquence représentative . . . 9
6.6 Temps moyen dans chaque état . . . 10
6.7 Tableau des transitions . . . 11
6.8 Distribution transversale . . . 11
6.9 Entropie transversale . . . 12
6.10 Descripteurs individuels . . . 13
6.11 Paramètres de visualisations . . . 13

7 Obtention de typologie par l'analyse de séquences **15**
7.1 Calcul des dissimilarités . . . 15
7.2 Clustering . . . 15
7.3 Cas multidimensionnel (multichannel) . . . 16

8 Autres analyses **19**
8.1 Analyse des divergences . . . 19
8.2 Analyse entre groupes . . . 21
8.3 Analyse de survie . . . 21
8.4 Reformatage pour analyses des transitions et des sous-séquences (CTA, SAMM, SHA) . . . 22

9 Autres types de séquences **25**

1 Introduction

L'objectif de ce document est de proposer une vue d'ensemble des possibilités qu'offrent les packages `TraMineR` et `TraMineRextras` en R pour l'analyse de séquences, et plus généralement de trajectoires. Ce document a été pensé pour être mis en parallèle avec son équivalent dédié à la librairie `TanaT` en Python, afin de comparer les deux outils et de déterminer le plus adapté selon la situation à traiter.

Pour faciliter cette comparaison, les deux documents suivent le même plan :

- Présentation des formats de données acceptés par les packages.
- Exploration des jeux de données intégrés et méthodes de simulation.
- Introduction des données "fil rouge" utilisées pour illustrer la suite du document.
- Manipulation des données et sélection des séquences selon différents critères.
- Description statistique et visualisation des séquences.
- Construction de typologies (calcul des dissimilarités et algorithmes de clustering), incluant le cas multidimensionnel (*multichannel*).
- Autres analyses (divergence, survie, test entre groupes, formatage pour autres analyses)
- Autres possibilités (séquences d'événements et séquences d'intervalles).

Nous traiterons dans ce document essentiellement de l'analyse de séquences d'**états**. Les autres types (événements et intervalles) seront spécifiquement abordés dans la dernière section.

2 Formats de données

Six formats de données sont proposés par `TraMineR` : **STS** (*successive states*), **SPS** (*successive state-duration spells*), **SPELL** (*vertical id-start-end-spells*), **DSS** (*distinct successive states*), **SRS** (*shifted replicated sequences*), **TSE** (*time stamped events*) et **SS** (*successive states*). De son côté, `TraMineRextras` permet d'étendre ces possibilités avec deux formats supplémentaires : **FCE** (*fixed column event*) et **HSPELL** (*horizontal spells*).

Prenons l'exemple de deux séquences X et Y, où a, b et c représentent les états possibles (l'alphabet) :

$$X = (a, a, a, b, b, c, c) \quad \text{et} \quad Y = (a, a, a, c, c, c, c)$$

Ces différents formats apportent plus ou moins d'informations sur les durées et les dates des différents états :

```
## [1] "--- Format STS ---"
```

```
##   T1 T2 T3 T4 T5 T6 T7
## X  a  a  a  b  b  c  c
## Y  a  a  a  c  c  c  c
```

```
## [1] " "
```

```
## [1] "--- Format SPS ---"
```

```
##   [1]     [2]     [3]     [4] [5] [6] [7]
## X "(a,3)" "(b,2)" "(c,2)" NA  NA  NA  NA
## Y "(a,3)" "(c,4)" NA      NA  NA  NA  NA
```

```
## [1] " "
```

```
## [1] "--- Format SPELL ---"
```

```
##   id begin end states
## 1  X     1   3      a
## 2  X     4   5      b
## 3  X     6   7      c
## 4  Y     1   3      a
## 5  Y     4   7      c
```

```
## [1] " "
```

```
## [1] "--- Format DSS ---"
```

```
##     [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1] "a"  "b"  "c"  NA   NA   NA   NA
## [2] "a"  "c"  NA   NA   NA   NA   NA
```

```
## [1] " "
```

```
## [1] "--- Format SRS ---"
```

```
##    id idx  T-6  T-5  T-4  T-3  T-2 T-1 T
## X   1   2 <NA> <NA> <NA> <NA> <NA>   a a
## X1  1   3 <NA> <NA> <NA> <NA>    a   a a
## X2  1   4 <NA> <NA> <NA>    a    a   a b
## X3  1   5 <NA> <NA>    a    a    a   b b
## X4  1   6 <NA>    a    a    a    b   b c
## X5  1   7    a    a    a    b    b   c c
## Y   2   2 <NA> <NA> <NA> <NA> <NA>   a a
## Y1  2   3 <NA> <NA> <NA> <NA>    a   a a
## Y2  2   4 <NA> <NA> <NA>    a    a   a c
## Y3  2   5 <NA> <NA>    a    a    a   c c
## Y4  2   6 <NA>    a    a    a    c   c c
## Y5  2   7    a    a    a    c    c   c c
```

```
## [1] " "
```

```
## [1] "--- Matrice de transition ---"
```

```
##   a     b     c
## a "a"   "a>b" "a>c"
## b "b>a" "b"   "b>c"
## c "c>a" "c>b" "c"
```

```
## [1] " "
```

```
## [1] "--- Format TSE ---"
```

```
##   id time event
## 1  1    0     a
## 2  1    3   a>b
## 3  1    5   b>c
## 4  2    0     a
## 5  2    3   a>c
```

```
## [1] " "
```

```
## [1] "--- Format SS ---"
```

```
##             seq
## X a-a-a-b-b-c-c
## Y a-a-a-c-c-c-c
```

```
## [1] " "
```

```
## [1] "--- Format FCE ---"
```

```
##   id a  b c
## 1  X 1  4 6
## 2  Y 1 NA 4
```

```
## [1] " "
```

```
## [1] "--- Format HSPELL ---"
```

```
##   begin1 end1 status1 begin2 end2 status2 begin3 end3 status3
## X      1    3       a      4    5       b      6    7       c
## Y      1    3       a      4    7       c     NA   NA    <NA>
```

Cependant, pour créer l'objet "séquence d'états" (*state sequence object*) indispensable à la réalisation des analyses, seuls les formats **STS**, **SPS** et **SPELL** sont acceptés.

```
seq_obj_sts <- seqdef(donnees_sts, informat="STS")
seq_obj_sps <- seqdef(donnees_sps, informat="SPS")
seq_obj_spell <- seqdef(donnees_spell, informat="SPELL")
seq_obj_ss <- seqdef(donnees_ss, 1)
```

Il est ensuite possible de passer d'un format à un autre à l'aide des fonctions de conversion `seqformat()`, `FCE_to_TSE()`, `TSE_to_STS()` et `HSPELL_to_STS()`. `TraMineRextras` permet également de convertir une séquences d'états au format **PersonPeriod** avec la fonction `toPersonPeriod()`.

Enfin, il est important de noter que la définition des séquences au sein du package gère nativement la notion de pondération, permettant ainsi d'**appliquer des poids aux séquences** le cas échéant.

3 Données exemples proposées et simulations

Le package `TraMineR` met à disposition plusieurs jeux de données pour illustrer ses fonctionnalités :

Jeu de données	Thématique	Format	Individus	Longueur	États	Covariables
actcal	Statut d'emploi (Swiss Household Panel)	STS ou TSE	2 000	12	4	11
biofam	Statut familial (Swiss Household Panel)	STS	2 000	16	8	7
mvad	Transition école-travail	STS	712	72	6	13
ex1	Pédagogique (données manquantes, poids)	STS	7	variable	4	-
ex2	Pédagogique (avec ou sans poids)	SS	100 ou 6	variable	4	-
famform	Historique familial	SS	5	variable	5	-

Aussi, il est important de noter que `TraMineR` ne propose pas de séquences comportant plusieurs domaines (multichannel).

Le package offre également la possibilité de simuler des données grâce à la fonction `seqgen()`. Elle permet de générer n séquences d'une taille donnée (`length`), en précisant l'`alphabet` souhaité et les probabilités associées à chaque élément.

```
# Exemple d'utilisation de la fonciton seqgen.
set.seed(42)
seqgen(n=3, length=8, alphabet = 1:3, p=c(0.2,0.1,0.7))
```

```
##   [1] [2] [3] [4] [5] [6] [7] [8]
## 1   2   2   3   1   3   3   1   3
## 2   3   1   3   1   2   3   3   2
## 3   2   3   3   3   2   3   2   2
```

4 Données fil-rouge

Pour réaliser cette comparaison systématique, nous utilisons les mêmes jeux de données dans les trois environnements. Pour cela, nous nous appuyons sur le Swiss Household Panel (SHP), une étude longitudinale d'envergure nationale lancée en 1999 qui recueille annuellement des informations sociodémographiques et économiques auprès d'un échantillon représentatif de ménages suisses. Pour les analyses de séquences unidimensionnelles, nous nous appuyons sur le jeu de données *Family life states* (`biofam`), notamment proposé comme jeu de données exemple par `TraMineR` et `sequenzo`. Pour les analyses nécessitant des séquences multidimensionnelles avec plusieurs thématiques, nous travaillons à partir d'un extrait plus large du Swiss Household Panel (`extract_shp`) que nous avons sélectionné nous-mêmes. Nous décrivons plus en détail ces deux jeux-de données ci-après.

##Family life states `biofam` Le jeu de données *Family life states* (`biofam`) est issu de l'enquête biographique rétrospective menée en 2002 dans le cadre du panel suisse de ménages. Il documente les trajectoires de formation familiale d'un échantillon de 2 000 individus sur une période de seize années, sans données manquantes, retraçant l'évolution de leur situation familiale de l'âge de 15 ans à l'âge de 30 ans.

Ces séquences sont initialement structurées au format **STS** et donc sont directement exploitables par `TraMineR`. Une version convertie au format **SPELL** est utilisée pour `TanaT`. L'alphabet des séquences décrites dans ce jeu de données est composé de huit statuts familiaux décrivant à la fois le lieu de résidence, le statut marital et la parentalité. Les individus sont ainsi catégorisés à chaque pas de temps selon qu'ils vivent au domicile parental (*Parent*), qu'ils l'ont quitté (*Left*), qu'ils ont un enfant (*Child*), qu'ils sont mariés (*Married*) qu'ils ont quitté le domicile familial et qu'ils sont mariés (*Left+Marr*), qu'ils ont quitté le domicile familial et ont un enfant (*Left+Child*), qu'ils soient hors du domicile familial, mariés et avec un enfant (*Left+Marr+Child*) ou qu'ils soient divorcés (*Divorced*).

En complément de la séquence d'états, le jeu de données inclut plusieurs covariables transversales: caractéristiques socio-démographiques de l'individu (sexe, année de naissance, première nationalité, langue de réponse au questionnaire)

et culturelles (religion et participation religieuse), ainsi que sur l'origine sociale avec le statut social du père et de la mère. Cependant, certaines de ces variables sont manquantes chez certains individus. Nous utilisons uniquement la variable sexe (disponible pour l'ensemble des individus) dans nos comparaisons.

Enfin, ce jeu de données propose deux sets de poids : l'un ajusté sur l'ensemble de la population suisse et l'autre ramené à la taille de l'échantillon. Cependant, ils ne sont fournis qu'à titre illustratif, puisque ce jeu de données est un sous-échantillon des réponses globales du panel et donc les poids proposés ne sont plus adaptés.

4.1 Extrait du Swiss Household Panel `extract_shp`

Le jeu de données `extract_shp` est un extrait des réponses à l'enquête longitudinale menée auprès d'un panel de ménages suisses entre 1999 et 2018 inclus. Nous avons sélectionné uniquement les ménages sans données manquantes et ayant participé à l'enquête chaque année, nous avons ainsi 20 observations de 1032 foyers. Trois thématiques ont été constituées, chacune regroupant une à trois variables évoluant au fil du temps, pour un total de six variables retenues.

Thématique familiale : Pour cette thématique, nous considérons uniquement le nombre de personnes composant le ménage. Cette variable prend des valeurs allant de 1 à 12, que nous regroupons en cinq catégories : 1, 2, 3, 4, et 5 habitants ou plus au sein du foyer.

Thématique résidentielle : Cette thématique regroupe trois variables : la région d'habitation, le type de commune d'habitation et le statut résidentiel. La région d'habitation comporte sept modalités, les acronymes des cantons correspondants sont indiqués entre parenthèses : Lac Léman (VD, VS, GE), Middleland (BE, FR, SO, NE, JU), Suisse Nord-Ouest (BS, BL, AG), Zurich, Suisse Est (GL, SH, AR, AI, SG, GR, TG), Suisse Centrale (LU, UR, SZ, OW, NW, ZG) et Tessin. Le type de commune d'habitation comporte quant à lui neuf modalités : centres, communes de banlieue, communes aisées, communes urbaines périphériques, communes touristiques, communes industrielles et tertiaires, communes rurales de banlieue, communes mixtes agricoles et communes agricoles périphériques. Le statut résidentiel se décline enfin en trois modalités : locataire, propriétaire ou copropriétaire, et logé à titre gratuit.

Thématique qualité de vie : Cette thématique comprend deux variables : l'état du lieu de vie du foyer et la satisfaction à l'égard de la situation financière du foyer. La première comporte trois modalités : en mauvais état, en bon état mais n'ayant pas été rénové récemment, et neuf ou récemment rénové. La seconde est à l'origine mesurée sur une échelle entière de 0 (pas du tout satisfait) à 10 (totalement satisfait), et est regroupée en cinq catégories : 0-2, 3-4, 5-6, 7-8 et 9-10.

Cette base de données est proposée au format **LONG**, qui n'est directement compatible avec aucun des trois packages. Une vesion au format **STS** est utilisée pour `TraMineR`.

Les variables retenues et les regroupements de modalités opérés pour ce comparatif ont été choisis arbitrairement dans le seul but d'illustrer les fonctionnalités des trois packages étudiés. Ce document n'a pas vocation à produire une analyse substantielle, et ces choix ne sauraient constituer une démarche pertinente dans un contexte applicatif réel.

```
# Téléchargement de biofam
data(biofam)

# Téléchargement des données SHP extraites
extract_shp <- read.csv("C:/Users/malnoeg/Documents/Stage_M2_SequenceAnalysis/Protocole/Code/data_SHP_sts.
shp_region <- extract_shp[c(2,3:22)]
shp_com2 <- extract_shp[c(2,23:42)]
shp_hh29 <- extract_shp[c(2,43:62)]
shp_nbpers <- extract_shp[c(2,63:82)]
shp_hh14 <- extract_shp[c(2,83:102)]
shp_hi30 <- extract_shp[c(2,103:122)]
```

5 Gestion des données

Pour l'analyse de séquence il peut se réveler très utile de filtrer et sélectionner des séquences sur certains critères comme la sélection d'un échantillon ou le respect d'une contrainte de temporelle. La librairie Python `TanaT` sépare le filtrage et la sélection des données en plusieurs catégories :

- **Query** : sélection par requête sur les états (ex. : présence de l'état A).
- **Static** : filtrage via les covariables des individus (ex. : sélection par genre).
- **Time** : restriction temporelle ou sélection sur une fenêtre de temps donnée (ex. : trajectoires entre 2001 et 2005).
- **Pattern** : recherche de motifs séquentiels précis (ex. : état A → état B → état D).
- **Length** : filtrage selon la longueur des séquences (ex. : taille strictement supérieure à 5).

Nous allons illuster ces différents filtrages par des exemples sur le jeu de données `biofam`.

Nous commençons tout d'abord par définir l'objet séquence avec la fonction `seqdef()` :

```
# Définition de l'objet séquence TraMineR
biofam.alphabet <- c(0,1,2,3,4,5,6,7)
biofam.labels <- c("Parent","Left","Married",
                   "Left+Marr","Child","Left+Child",
                   "Left+Marr+Child","Divorced")
biofam.scode <- c("P", "L", "M", "LM", "C","LC","LMC","D")
biofam.seq <- seqdef(biofam,
                     var = 10:25,
                     alphabet = biofam.alphabet,
                     states = biofam.scode,
                     labels = biofam.labels)
head(biofam.seq, 5)
```

```
##      Sequence
## 1167 P-P-P-P-P-P-P-P-P-LM-LMC-LMC-LMC-LMC-LMC-LMC
## 514  P-L-L-L-L-L-L-L-L-L-L-LM-LMC-LMC-LMC-LMC
## 1013 P-P-P-P-P-P-P-L-L-L-L-L-LM-LMC-LMC-LMC
## 275  P-P-P-P-P-L-L-L-L-L-L-L-L-L-L-L
## 2580 P-P-P-P-P-L-L-L-L-L-L-L-L-LMC-LMC-LMC
```

5.1 Query

Exemple : ne sélectionner que les séquences passant au moins une fois par l'état "L" (Left) au cours du temps.

```
indexL <- rowSums(biofam.seq == "L", na.rm = TRUE) > 0
head(biofam.seq[indexL, ], 5)
```

```
##      Sequence
## 514  P-L-L-L-L-L-L-L-L-L-L-LM-LMC-LMC-LMC-LMC
## 1013 P-P-P-P-P-P-P-L-L-L-L-L-LM-LMC-LMC-LMC
## 275  P-P-P-P-P-L-L-L-L-L-L-L-L-L-L-L
## 2580 P-P-P-P-P-L-L-L-L-L-L-L-L-LMC-LMC-LMC
## 47   P-P-P-P-P-P-L-L-L-L-L-LM-LM-LM-LMC-LMC
```

5.2 Static

Exemple : sélection des personnes nées strictement après 1950.

```
index1950 <- biofam["birthyr"]>1950
head(biofam.seq[index1950, ], 5)
```

```
##      Sequence
## 1990 P-P-P-P-P-P-P-P-LM-LM-LM-LM-LM-LM-LM-LM
## 2193 P-P-P-P-P-P-P-P-P-P-P-P-P-LM-LM-LM
## 1571 P-P-P-P-L-L-L-LM-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC
## 1917 P-P-P-P-P-P-P-P-LM-LM-LM-LMC-LMC-LMC-LMC-LMC
## 1102 P-L-L-L-L-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC
```

5.3 Time

Exemple : sélectionner les séquences comprises (en entiereté) entre le temps 1 et le temps 8.

```
longueurs <- seqlength(biofam.seq[2:16])
head(biofam.seq[longueurs <= 8, 2:16], 5)
```

```
##      Sequence
```

Dans `biofam`, toutes les séquences sont de taille 16, nous obtenons donc un objet séquence vide.

5.4 Pattern

Exemple : présence du pattern Parent -> Left + Married.

```
seq_texte <- seqconc(biofam.seq)
indexPattern <- grepl("P-LM-", seq_texte)
head(biofam.seq[indexPattern, ], 5)
```

```
##      Sequence
## 1167 P-P-P-P-P-P-P-P-P-LM-LMC-LMC-LMC-LMC-LMC-LMC
## 1990 P-P-P-P-P-P-P-P-LM-LM-LM-LM-LM-LM-LM-LM
## 2136 P-P-P-P-P-P-P-P-LM-LM-LM-D-D-D-D-D
## 2193 P-P-P-P-P-P-P-P-P-P-P-P-P-LM-LM-LM
## 1989 P-P-P-P-LM-LM-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC-LMC
```

5.5 Length

Exemple : sélection des séquences de longueur strictement supérieure à 10.

```
longueurs <- seqlength(biofam.seq)
head(biofam.seq[longueurs > 10, ], 5)
```

```
##      Sequence
## 1167 P-P-P-P-P-P-P-P-P-LM-LMC-LMC-LMC-LMC-LMC-LMC
## 514  P-L-L-L-L-L-L-L-L-L-LM-LMC-LMC-LMC-LMC
## 1013 P-P-P-P-P-P-P-L-L-L-L-L-LM-LMC-LMC-LMC
## 275  P-P-P-P-P-L-L-L-L-L-L-L-L-L-L
## 2580 P-P-P-P-P-L-L-L-L-L-L-L-L-LMC-LMC-LMC
```

5.6 Changement de granularité

Dans le même esprit de préparation des données, il est parfois utile de procéder à un **changement de granularité**, par exemple dans un contexte où les différents individus n'auraient pas répondu à la même fréquence mais que l'on souhaite les étudier ensemble.

Exemple : aggrégation des états 4 par 4, au lieu de 16 états en choisissant l'état le plus fréquent.

```
head(seqgranularity(biofam.seq, tspan = 4, method = "mostfreq"), 5)
```

```
##   Sequence
## 1 P-P-LMC-LMC
## 2 L-L-L-LMC
## 3 P-P-L-LMC
## 4 P-L-L-L
## 5 P-L-L-LMC
```

6 Descriptions des séquences et visualisations

Le package `TraMineR` met à disposition un évantail d'outils statistiques et graphiques pour explorer les trajectoires de manière globale ainsi que pour chaque séquence individuelle.

6.1 Alphabet

L'alphabet qui décrit les séquences s'obtient via la fonciton `alphabet()`.

```
alphabet(biofam.seq)
```

```
## [1] "P"   "L"   "M"   "LM"  "C"   "LC"  "LMC" "D"
```

6.2 Visualisation des séquences

L'observation brute des trajectoires s'effectue via `seqiplot()`, qui est adaptée pour la visualisation détaillée de quelques séquences. Pour les échantillons de taille plus grande, `seqIplot()` est privilégié et est principalement utile

pour l'observation de tendances globales plutôt que la distinction de chaque cas individuel.

```
# Visualisation de l'ensemble des séquences (quelques indices)
seqiplot(biofam.seq, idx = 10:20,
         with.legend = "right", cex.axis=0.7, cex.legend=1)
```

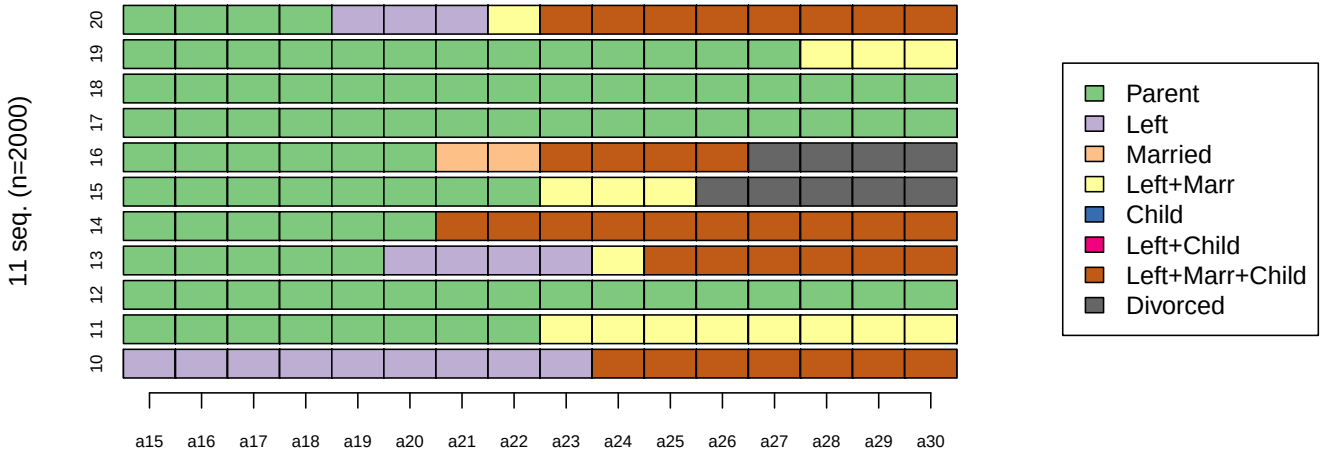


Figure 1: Visualisation des séquences 10 à 20 du jeu de données biofam

```
# Visualisation de l'ensemble des séquences (global)
seqIplot(biofam.seq, border = NA,
         with.legend = "right", cex.axis=0.8, cex.legend=1)
```

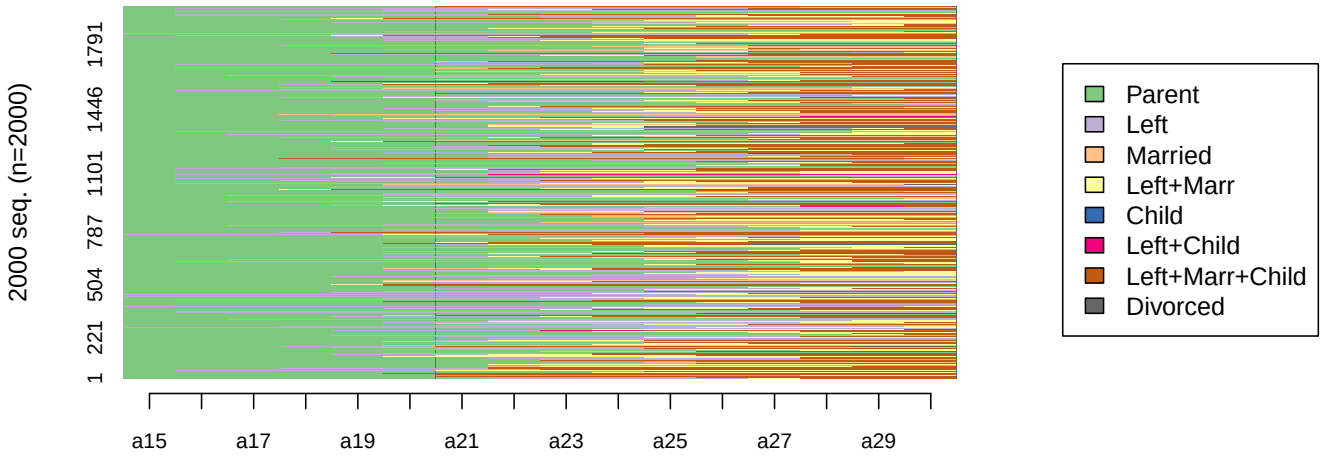


Figure 2: Visualisation de l'ensemble des séquences du jeu de données biofam

6.3 Fréquences des séquences

L'identification des séquences les plus fréquentes est gérée par la fonction `seqtab()` et peut être illustrée par la fonction `seqfplot()` avec les fréquences cumulées.

```
# Fréquences des séquences et visualisation
head(seqtab(biofam.seq), 3)
```

```
##          Freq Percent
## P/16      154     7.7
## P/6-L/10   45     2.2
## P/5-L/11   40     2.0
```

```
seqfplot(biofam.seq, with.legend = "right", cex.axis=0.8, cex.legend=1)
```

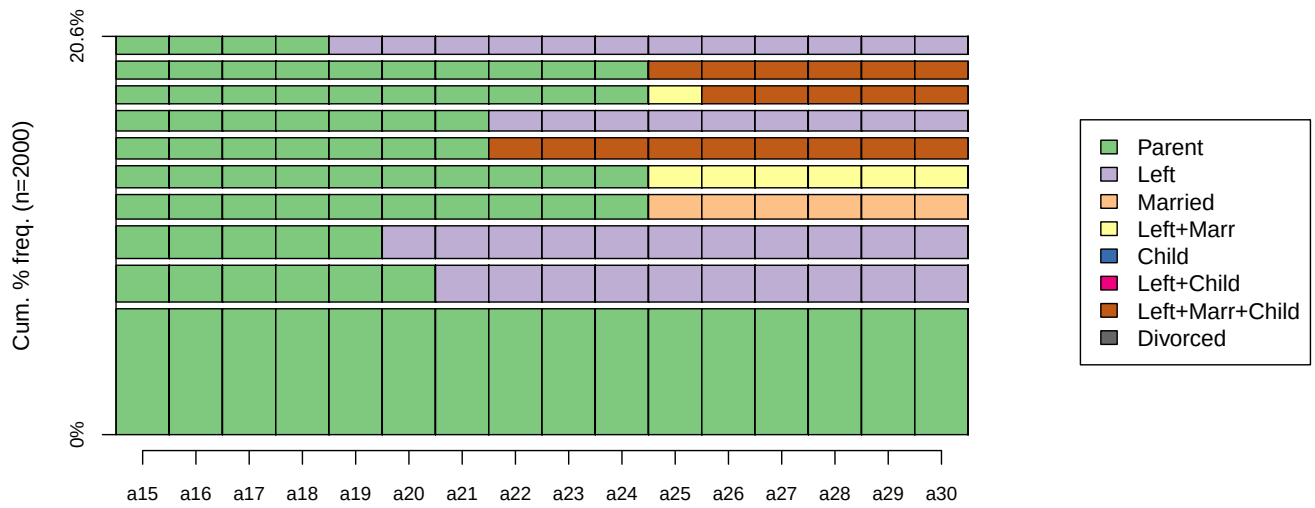


Figure 3: Fréquences cumulatives des séquences de biofam

6.4 Fréquences des états co-occurents

L'identification des combinaisons d'états co-occurents les plus fréquents au sein des séquences est réalisé via la fonction `seqtabstocc()`.

```
seqtabstocc(biofam.seq)
```

```
##             Freq Percent
## LM-LMC-P     293   14.65
## L-P          270   13.50
## L-LM-LMC-P   259   12.95
## M-P          205   10.25
## LM-P         186    9.30
## L-LM-P       178    8.90
## LMC-P        170    8.50
## P            154    7.70
## L-LMC-P      117    5.85
## D-LM-P        21    1.05
```

6.5 Séquence représentative

Dans la même logique, la séquence modale (la séquence la plus représentative) s'extrait via `seqmodst()` et peut-être visualisée avec `seqmsplot()`.

```r
# Séquence représentative
seqmodst(biofam.seq)
```

```
##   [Modal state sequence]
##   Sequence
## 1 P-P-P-P-P-P-P-P-P-L-LMC-LMC-LMC-LMC-LMC
##
##   [State frequencies]
##        a15  a16  a17  a18  a19  a20 a21 a22  a23  a24  a25  a26  a27  a28 a29
## Freq. 0.99 0.95 0.92 0.88 0.82 0.71 0.6 0.5 0.42 0.34 0.26 0.26 0.31 0.36 0.4
##        a30
## Freq. 0.44
```

```r
seqmsplot(biofam.seq, with.legend = "right", cex.axis=0.8, cex.legend=1)
```

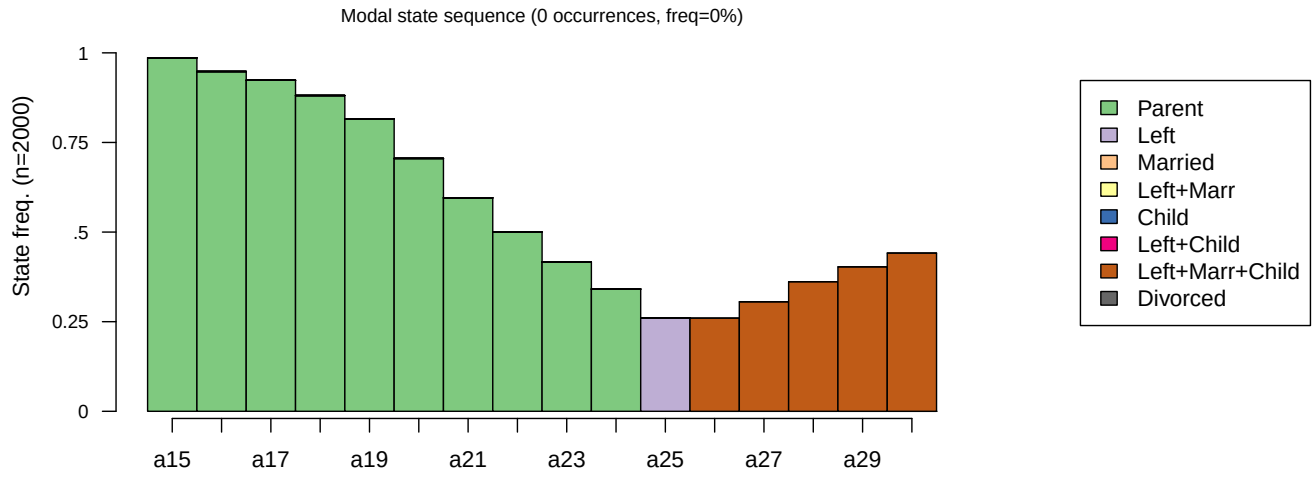


Figure 4: Séquence représentative de biofam

6.6 Temps moyen dans chaque état

Pour analyser les durées moyennes passées dans chaque état, La fonction `seqmeant()` permet d'obtenir les durées moyennes passées dans chaque état et son résultat est illustré dans un barplot avec `seqmtplot()`.

```r
# Temps moyen dans chaque état
seqmeant(biofam.seq)
```

```
##       Mean
## P   8.0280
## L   2.9440
## M   0.7330
## LM  1.6220
## C   0.0095
## LC  0.0835
## LMC 2.4190
## D   0.1610
```

```r
seqmtplot(biofam.seq, border = NA,
          with.legend = "right", cex.axis=0.8, cex.legend=1)
```

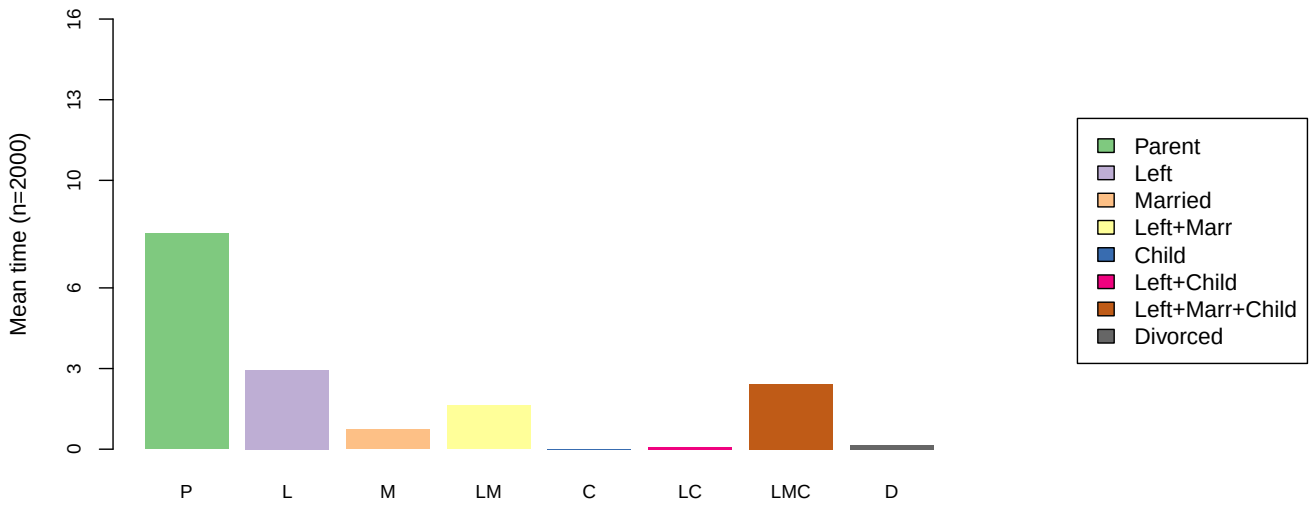


Figure 5: Barplot du temps moyen passé dans chaque état

6.7 Tableau des transitions

Pour l'analyse des dynamiques de transitions entre les états, le package propose la fonction `seqtrate()` qui renvoie la matrice des transitions.

```
# Tableau des transitions
round(seqtrate(biofam.seq), 3)
```

```
##           [-> P] [-> L] [-> M] [-> LM] [-> C] [-> LC] [-> LMC] [-> D]
## [P ->]     0.886  0.055  0.015   0.032  0.000   0.001    0.011  0.000
## [L ->]     0.000  0.890  0.000   0.083  0.000   0.004    0.023  0.000
## [M ->]     0.000  0.000  0.969   0.010  0.000   0.000    0.011  0.010
## [LM ->]    0.000  0.000  0.000   0.787  0.000   0.000    0.199  0.014
## [C ->]     0.000  0.000  0.125   0.000  0.812   0.062    0.000  0.000
## [LC ->]    0.000  0.000  0.000   0.000  0.000   0.882    0.118  0.000
## [LMC ->]   0.000  0.000  0.000   0.000  0.000   0.000    0.994  0.006
## [D ->]     0.000  0.000  0.000   0.000  0.000   0.000    0.000  1.000
```

6.8 Distribution transversale

Une illustration de l'évolution des distributions transversale (chronogramme) est proposé par les fonctions `seqstatd()` et la fonction `seqdplot()`.

```
# Distribution transversale
distribution_transversale <- seqstatd(biofam.seq)$Frequencies
head(round(distribution_transversale, 2), 3)
```

```
##    a15  a16  a17  a18  a19  a20  a21  a22  a23  a24  a25  a26  a27  a28  a29
## P 0.99 0.95 0.92 0.88 0.82 0.71 0.60 0.50 0.42 0.34 0.25 0.20 0.16 0.13 0.10
## L 0.01 0.05 0.07 0.11 0.15 0.23 0.28 0.30 0.29 0.28 0.26 0.23 0.20 0.17 0.16
## M 0.00 0.00 0.00 0.00 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.09 0.10
##    a30
## P 0.08
## L 0.14
## M 0.10
```

```
seqdplot(biofam.seq, with.legend = "right", cex.axis=0.8)
```

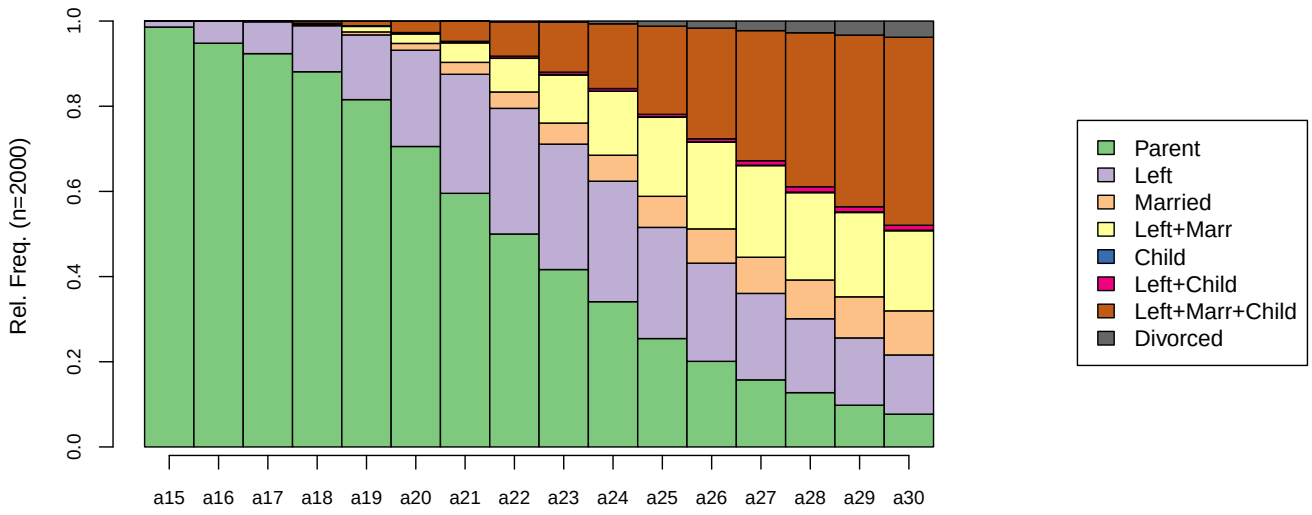


Figure 6: Chronograme des séquences de biofam

6.9 Entropie transversale

L'évolution temporelle de l'hétérogénéité des états de l'échantillon peut être mesurée par l'entropie transversale avec `seqHtplot()`, qu'il est possible de superposer au chronogramme grâce à la fonction `seqHplot()`.

```
# Fonction entropie
seqHtplot(biofam.seq,cex.axis=0.8)
```

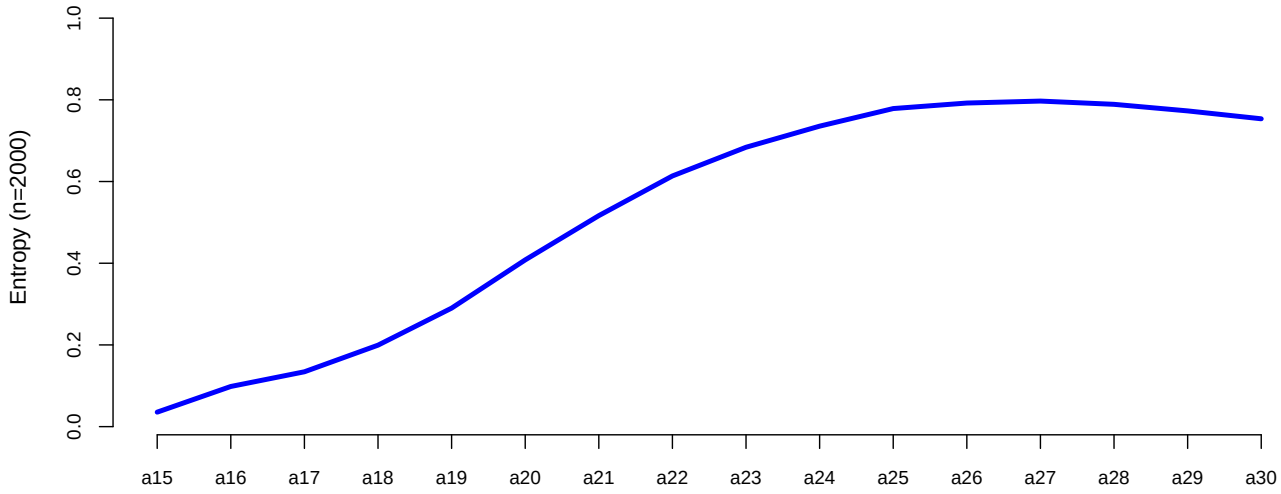


Figure 7: Entropie transversale de biofam

```
# Fonction entropie superposée au graphe de distribution
seqdHplot(biofam.seq, with.legend = "right",cex.axis=0.8)
```

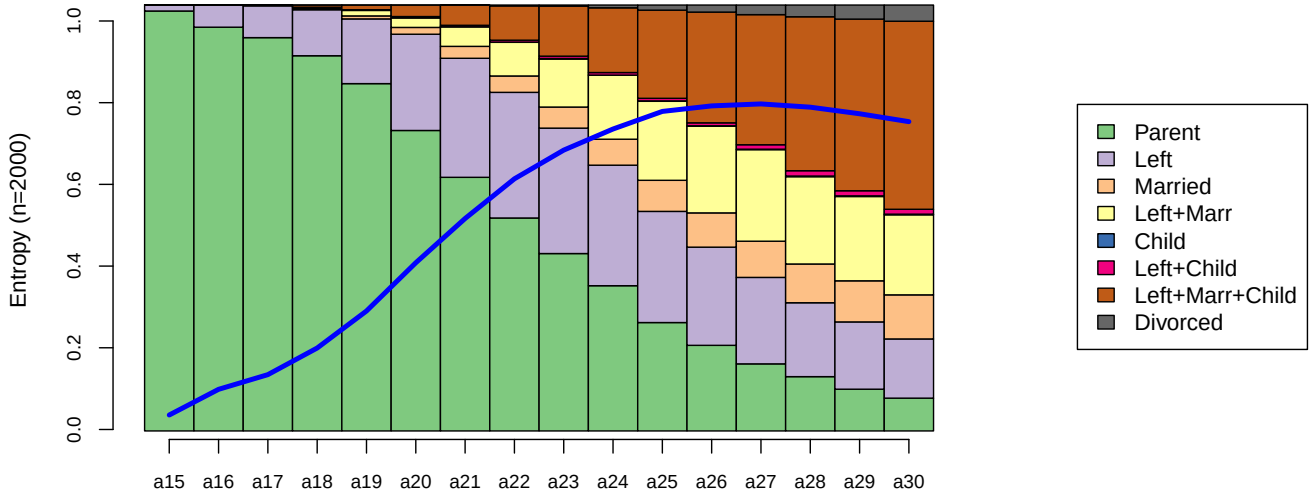


Figure 8: Entropie transversale superposée au chronogramme de biofam

6.10 Descripteurs individuels

Au niveau individuel, plusieurs descripteurs sont à disposition pour décrire chaque séquence selon différents aspects : le nombre de sous-séquences qui la compose (`seqsubsn()`), sa longueur (`seqlength()`), les états distincts qui la compose (`seqdss()`), son entropie propre (`seqient()`), sa complexité (`seqici()`), sa turbulence (`seqST()`), sa volatilité (`seqivolatility()`), un indicateur de la proportion d'états positifs ou négatifs selon des listes passées en argument (`seqipos()`), son potentiel intégratif (`seqintegr()`), sa badness (`seqibad()`), sa degradation (`seqidegrad()`), sa précarité (`seqprecarity()`) et son insécurité (`seqinsecurity()`).

```
# Descripteurs individuels
head(seqsubsn(biofam.seq), 5) # nombre de sous-séquences
head(seqlength(biofam.seq),5) # longueur
head(seqdss(biofam.seq), 5) # états disctincts
head(seqient(biofam.seq), 5) # entropie within
head(seqici(biofam.seq), 5) # complexity
head(seqST(biofam.seq), 5) # turbulence
head(seqivolatility(biofam.seq), 5) # Volatilité
head(seqipos(biofam.seq,
             pos.states = c("P","L","LM","LC","LMC","M","C"),
             neg.states=c("D")),5) # Post-Negative
head(seqintegr(biofam.seq), 5) # Integration
head(seqibad(biofam.seq), 5) # Badness
head(seqidegrad(biofam.seq), 5) # Degradation
head(seqprecarity(biofam.seq), 5) # Precarité
head(seqinsecurity(biofam.seq), 5) # Insecurité
```

6.11 Paramètres de visualisations

Pour l'ensemble des visualisations, il est possible de préciser en argument `group=...` pour obtenir des visualisations différentes pour chaque facteur d'une variable catégorielle.

```
seqIplot(biofam.seq, group=biofam$sex, border = NA,
          with.legend = "right", cex.axis=0.8, cex.legend=1.7, cex.lab=1.7)
```

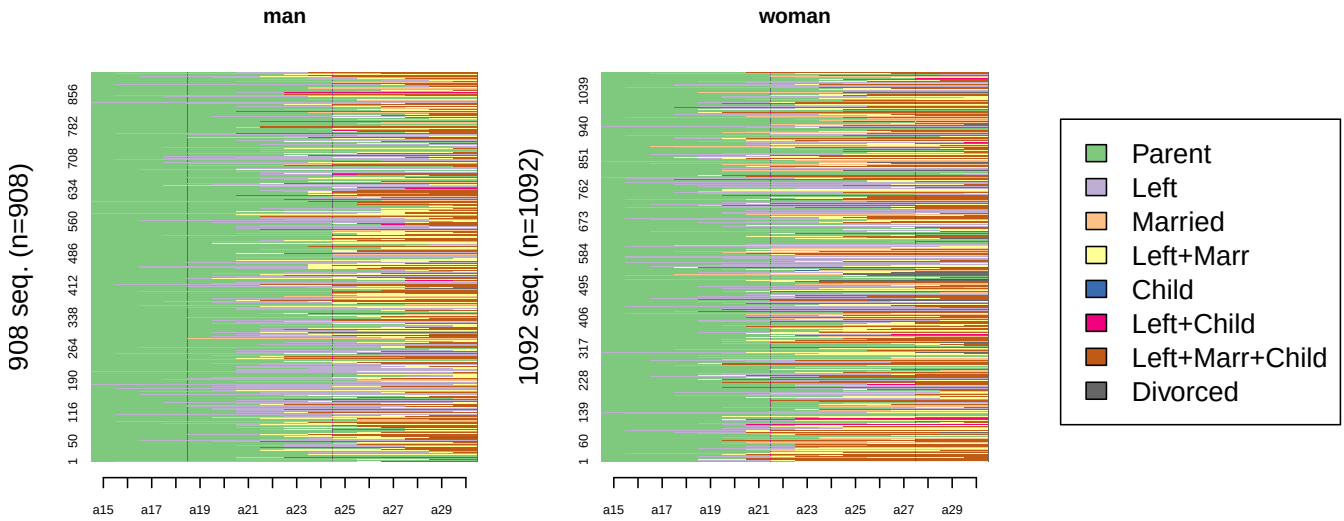


Figure 9: Chronogramme des séquences en fonction du sexe

Il est également possible de prendre en compte les poids de sondage du jeu de données s'ils sont précisés en modifiant l'épaisseur des séquences pour les visualisations et en les prennant en compte pour les statistiques.

```
biofam.seq_w <- seqdef(biofam,
                       var = 10:25,
                       alphabet = biofam.alphabet,
                       states = biofam.scode,
                       labels = biofam.labels,
                       weights = biofam$wp00tbgs)
seqiplot(biofam.seq_w, idx = 10:20, with.legend = "right", cex.axis=0.7)
```

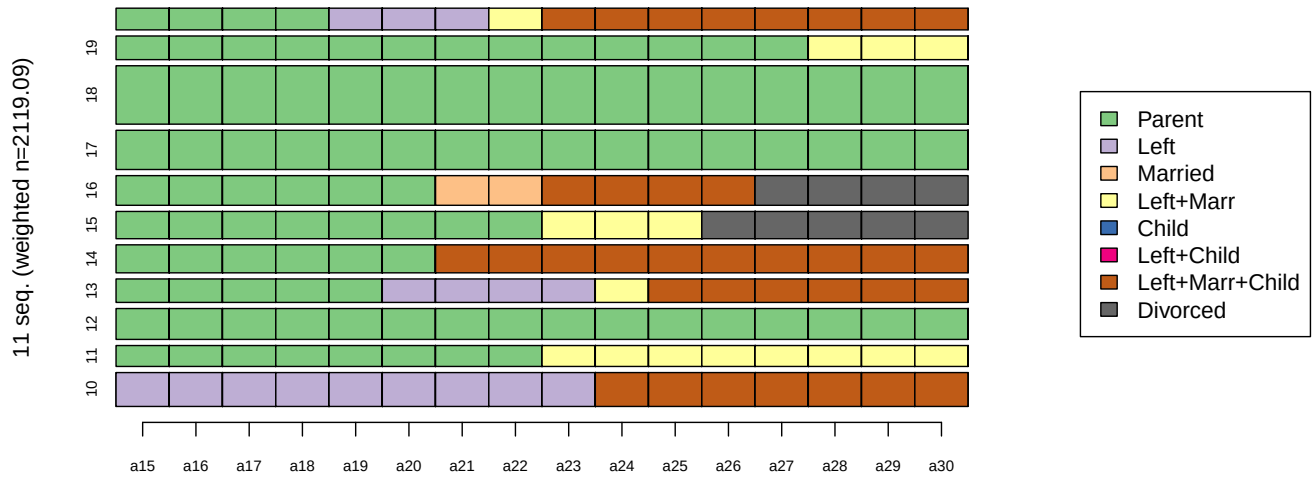


Figure 10: Visualisation des séquences 10 à 20, épaisseur selon le poids de la séquence dans le jeu de données

Enfin, l'ensemble de ces statistiques et visualisations peuvent être paramétré pour afficher des labels ou des couleurs spécifiques (`cpal()`).

7 Obtention de typologie par l'analyse de séquences

L'une des applications majeure de l'analyse de séquence est la construction de typologie : le regroupement des séquences partageant des propriétés similaires. L'identification de ces profils types permet ensuite de travailler avec des covariables afin d'expliquer la formation de ces différents groupes.

La construction de ces typologies se fait en deux étapes : l'évaluation des écarts entre chaque paire de séquences qui fournit une matrice de (di)similairités, puis l'application d'un algorithme de classification (clustering) sur cette matrice afin de former des groupes d'individus similaires (ou peu dissimilaires).

Pour chacune de ces étapes, le package `TraMineR` propose plusieurs possbilités que nous allons illustrer sur le jeu de données `biofam`.

7.1 Calcul des dissimilarités

Pour la première étape de ce processus, avec la fonction `seqdist()`, `TraMineR` offre un large éventail de métriques permettant de quantifier les différences entre les séquences, et sur plusieurs aspects :

- Distances d'édition : OM, OMlo, OMslen, OMspell, OMstran, Hamming, Dynamic Hamming et TWED.
- Mesures basées sur le nombres d'attributs communs : Longuest Common Subsequence (LCS), Longest Common Prefix (LCP), Longuest Common Suffix (RLCP), Number of Matching Subsequences (NMS) et NMSMSTet SVRspell.
- Distances entre les distributions d'états : Euclide et χ^2 (CHI2)

Pour notre jeu de données, nous faisons le choix d'appliquer l'algorithme d'Optimal Matching (OM).

```
biofam.om <- seqdist(biofam.seq, method = "OM", indel=1, sm = "CONSTANT")
```

```
## [1] "0.215  s"
```

7.2 Clustering

Une fois la matrice de dissimilarités calculée, la seconde étape consiste à regrouper les trajectoires similaires au moyen d'un algorithme de clustering. Néanmoins, `TraMineR` ne met pas à disposition d'algorithme de clustering mais propose de faire appel aux algorithmes déjà implémentés en R, notamment au travers du package `cluster`. Celui ci propose, entre autres, les algorithmes suivants :

- Clustering Large Application (CLARA)
- Partitioning Around Medoids (PAM)
- hierarchical clustering (agglomerative nesting, AGNES)
- Fuzzy ANalYsis clustering (FANNY).

Le package `TraMineR` est compatible avec tout algorithme qui se base sur une matrice de dissimilarité et renvoie un vecteur contenant le cluster de chaque séquence, ainsi il est possible d'utiliser d'autres packages ou de coder son propre algorithme.

Pour notre jeu de données, nous choisissons d'appliquer un algorithme de clustering hiérarchique en retenant 2 clusters.

```
clusterward <- agnes(biofam.om, diss = TRUE, method = "ward")
nb_clust <- 2
biofam.clust <- cutree(clusterward, k=nb_clust)
clust.lab <- factor(biofam.clust, labels = paste("Cluster",1:nb_clust))
```

```
## [1] "0.199  s"
```

```
seqdplot(biofam.seq, group = clust.lab, border = NA,
         with.legend = "right", cex.legend=2)
```

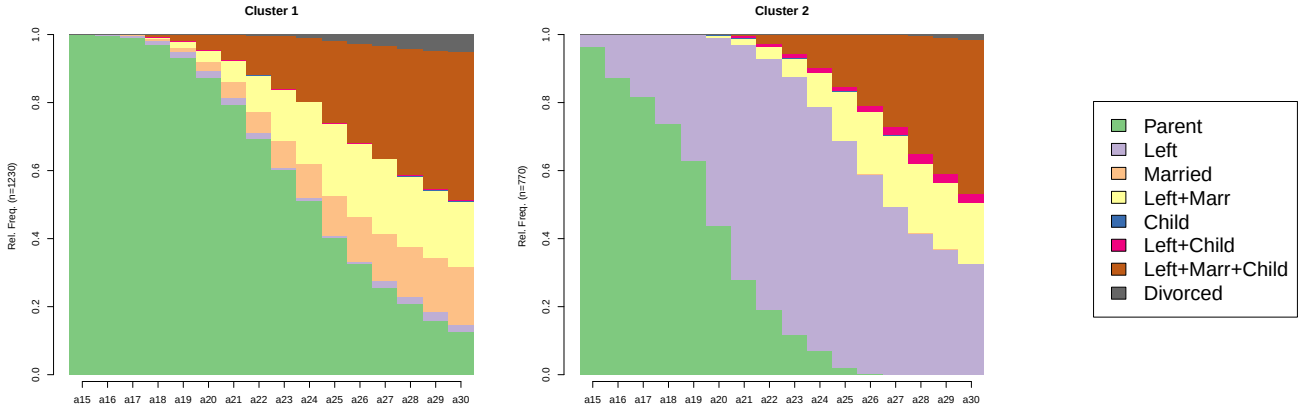


Figure 11: Chronogramme selon le cluster

```
seqIplot(biofam.seq, group = clust.lab, border = NA, with.legend = "right", cex.legend=2)
```

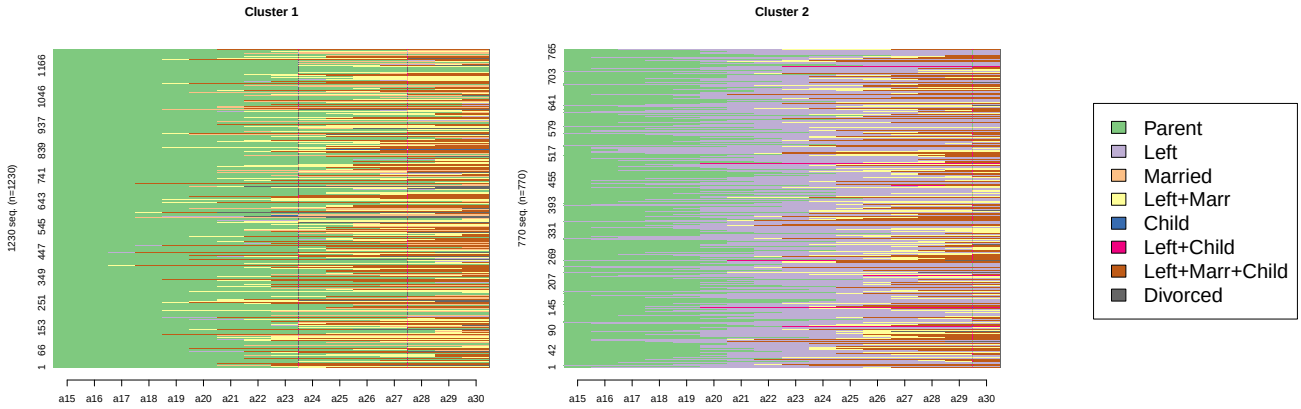


Figure 12: Visualisation de l'ensemble des séquences selon le cluster

Les outils de visualisations selon un groupe directement intégré dans les foncitons de visualisation de `TraMineR` sont très utiles pour pouvoir facilement visualiser les résultats de l'analyse des typologies et de voir aisément sur quelles propriétés les typologies se différencient.

7.3 Cas multidimensionnel (multichannel)

L'analyse de séquences ne se limite pas qu'au cas unidimensionnel et peut également être intéressante à appliquer dans un contexte multidimensionnel comme celui du modèle SaLTo présenté en introduction. Le processus analytique dans le cadre multidimensionnel reste sensiblement le même : il s'agit toujours de définir des séquences à partir d'un certain alphabet d'états, de choisir le coût du passage d'un état à un autre, de calculer la dissimilarité entre les séquences et d'appliquer un algorithme de clustering pour obtenir des groupements de séquences similaires. L'enjeu devient alors de choisir le moment auquel considérer l'aspect multidimensionnel. Quatre approches sont distinguées dans la littérature : l'extension de l'alphabet (EA, *Extended Alphabet*), l'association des matrices de coût (CAT, *Cost Additive Trick*), l'association des matrices de dissimilarité (DAT, *Distance Additive Trick*) et la combinaison des clusters (CombT, *Combining Trajectories*) qui sont toutes implémentables en R avec `TraMineR`.

7.3.1 Extended Alphabet

```
start.time <- Sys.time()
# Construction d'un jeu de données avec l'alphabet combiné
  # Initiatilisation du jeu de données final
combined_data <- data.frame(idhous = shp_region$idhous)
 # Pour chaque année, on extrait la valeur de chaque variable
for (i in 1:18) {
  v1 <- shp_region[, i + 1]
```

```
  v2 <- shp_com2[, i + 1]
  v3 <- shp_hh29[, i + 1]
  v4 <- shp_nbpers[, i + 1]
  v5 <- shp_hh14[, i + 1]
  # Combinaison et sauvegarde dans le jeu de données final
  combined_data[, paste0("T", i)] <- paste(v1, v2, v3, v4, v5, sep = "/")
}
# Définition de l'alphabet et de l'objet STS
alphabet_extended <- sort(unique(as.vector(as.matrix(combined_data[, -1]))))
shp.ea <- seqdef(combined_data[, -1],
                      alphabet = alphabet_extended,
                      id = combined_data$idhous)
EA_distance <- seqdist(shp.ea, method = "OM", indel=1, sm = "CONSTANT")

# Clustering avec le package cluster
clusterward <- agnes(EA_distance, diss = TRUE, method = "complete")
nb_clust <- 4
cat.clust <- cutree(clusterward, k=nb_clust)
clust.lab <- factor(cat.clust, labels = paste("Cluster",1:nb_clust))
end.time <- Sys.time()
time.taken <- end.time - start.time
print(paste(time.taken," s"))
```

```
## [1] "1.84110617637634  s"
```

7.3.2 Cost Additive Trick

```
start.time <- Sys.time()
# Création des objets de séquence pour chaque variable
seq_v1 <- seqdef(shp_region[, -1])
seq_v2 <- seqdef(shp_com2[, -1])
seq_v3 <- seqdef(shp_hh29[, -1])
seq_v4 <- seqdef(shp_nbpers[, -1])
seq_v5 <- seqdef(shp_hh14[, -1])

# Calcul sur plusieurs canaux
channels <- list(seq_v1, seq_v2, seq_v3, seq_v4,seq_v5)
CAT_distance <- seqMD(channels,
                      method="OM",
                      sm =list("CONSTANT","CONSTANT",
                               "CONSTANT","CONSTANT", "CONSTANT"),
                      what="diss")

# Clustering
clusterward <- agnes(CAT_distance, diss = TRUE, method = "complete")
nb_clust <- 4
cat.clust <- cutree(clusterward, k=nb_clust)
clust.lab <- factor(cat.clust, labels = paste("Cluster",1:nb_clust))
end.time <- Sys.time()
time.taken <- end.time - start.time
print(paste(time.taken," s"))
```

```
## [1] "11.0738959312439  s"
```

7.3.3 Distance Additive Trick

```
start.time <- Sys.time()
# Définition des objets de séquence (si non fait)
seq_v1 <- seqdef(shp_region[, -1])
seq_v2 <- seqdef(shp_com2[, -1])
seq_v3 <- seqdef(shp_hh29[, -1])
seq_v4 <- seqdef(shp_nbpers[, -1])
seq_v5 <- seqdef(shp_hi30[, -1])
```

```r
# Calcul des matrices de distances (ex: Optimal Matching)
# On peut utiliser des méthodes ou des coûts de substitution différents par canal
dist1 <- seqdist(seq_v1, method = "OM", sm = "CONSTANT")
dist2 <- seqdist(seq_v2, method = "OM", sm = "CONSTANT")
dist3 <- seqdist(seq_v3, method = "OM", sm = "CONSTANT")
dist4 <- seqdist(seq_v4, method = "OM", sm = "CONSTANT")
dist5 <- seqdist(seq_v5, method = "OM", sm = "CONSTANT")

# Combinaison des deux distance
weights <- c(1,1,1,1,1)
DAT_distance <- weights[1]* dist1 + weights[2]* dist2 +
  weights[3]* dist3 + weights[4]* dist4 +
  weights[5]* dist5

# CLuster
library("cluster")
clusterward <- agnes(DAT_distance, diss = TRUE, method = "complete")
nb_clust <- 4
cat.clust <- cutree(clusterward, k=nb_clust)
clust.lab <- factor(cat.clust, labels = paste("Cluster",1:nb_clust))
end.time <- Sys.time()
time.taken <- end.time - start.time
print(paste(time.taken," s"))
```

```
## [1] "2.0573308467865  s"
```

7.3.4 Combinin Trajectories

```r
start.time <- Sys.time()
# Liste pour stocker les labels de clusters
cluster_labels <- list()
# Création des objets de séquence pour chaque variable
seq_v1 <- seqdef(shp_region[, -1])
seq_v2 <- seqdef(shp_com2[, -1])
seq_v3 <- seqdef(shp_hh29[, -1])
seq_v4 <- seqdef(shp_nbpers[, -1])
seq_v5 <- seqdef(shp_hh14[, -1])
seq_list = list(seq_v1,seq_v2,seq_v3,seq_v4,seq_v5)
# Exemple de boucle pour traiter chaque canal (en supposant 5 variables)
for (i in 1:5) {
  # 1. Calcul de la distance pour le canal i
  d_matrix <- seqdist(seq_list[[i]], method = "OM", sm = "TRATE")
  # 2. Clustering (Ward)
  clusterward <- agnes(DAT_distance, diss = TRUE, method = "complete")
  # 3. Découpage en k groupes (le nombre k peut varier selon le canal)
  # Ici, nous fixons k=4 par défaut à titre d'exemple pour tous les canaux
  nb_clust <- 4
  cluster_labels[[i]] <- cutree(clusterward, k=nb_clust)
}

# Combinaison des 5 clusters pour former des sous-clusters croisés
# Fusion des labels en une chaîne de caractères unique par individu
clust.lab_global <- paste(cluster_labels[[1]],
                          cluster_labels[[2]],
                          cluster_labels[[3]],
                          cluster_labels[[4]],
                          cluster_labels[[5]],
                          sep = "_")
# Conversion en facteur pour analyse ultérieure
combined_factor <- as.factor(clust.lab_global)
time.taken <- Sys.time() - start.time
print(paste(time.taken," s"))
```

```
## [1] "4.52519798278809  s"
```

7.3.5 Commentaire sur l'implémentation proposée

Il faut toutefois souligner une limite pratique quant à l'implémentation avec `TraMineR` des analyses multidimensionnel. Si les objets du packages permettent de réaliser ces analyses, le package ne dispose pas nativement d'une classe d'objet unifiée permettant de modéliser globalement une trajectoire avec plusieurs cannaux. En l'absence d'une fonction centralisatrice, l'utilisateur doit jongler lui-même entre les différents cannaux et matrices, ce qui est envisageable pour un nombre restreint de cannaux, mais moins lorsque leur nombre augmente fortement.

8 Autres analyses

Outre l'exploration des typologie au sein des séquences, `TraMineR` propose d'autres méthodes d'analyses comme l'analyse des divergences (*discrepancy analysis*), des tests de comparaisons des séquences entre différentes sous-populations ainsi que de l'analyse de survie.

Nous allons illustrer l'implémentation de ces possibilités avec le jeu de données `biofam`.

8.1 Analyse des divergences

Le package `TraMineR` permet de mesurer la divergence d'une ensemble de séquence à partir de la matrice de dissimilarité avec `dissvar()`.

```
# Compute the discepancy
dissvar(biofam.om)
```

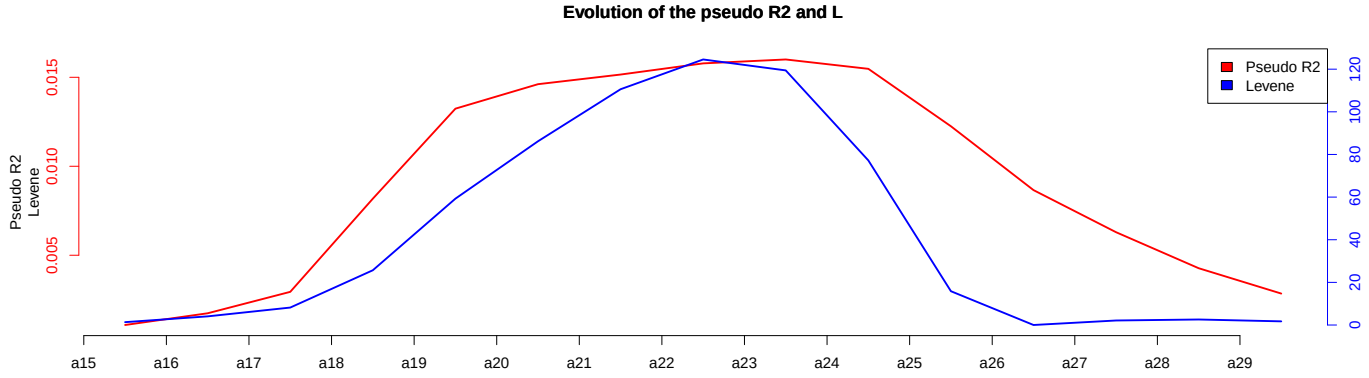
```
## [1] 7.909014
```

Egalement, il est possible de réaliser une analyse de la divergence selon une variable catégorielle et de tester statistiquement l'association entre la variable sélectionnée et la matrice de dissimilarités avec la fonction `dissassoc()` pour un point de vue global et `seqdiff()` pour une analyse à chaque pas de temps. Nous l'illustrons avec la variable sexe du jeu de données `biofam`.

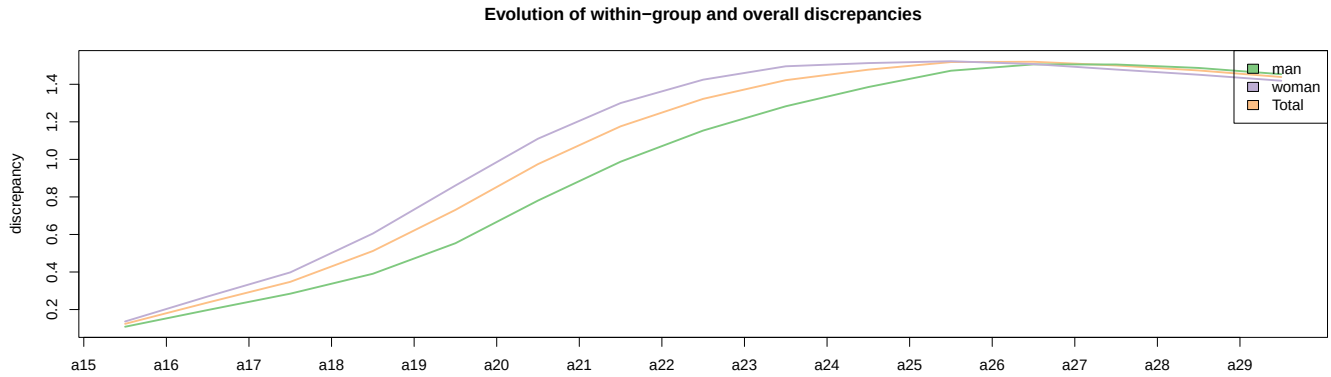
```
# Explication avec une seule variable / groupement
# Statistiques et p-values (Pseudo F, Pseudo Fbf, Pseudo R2, Bartlett, and Levene))
da <- dissassoc(biofam.om, group=biofam$sex, R=5000)
print(da$stat)
```

```
##                      t0 p.value
## Pseudo F   18.870242380   2e-04
## Pseudo Fbf 19.092235332   2e-04
## Pseudo R2   0.009356201   2e-04
## Bartlett    3.338519343   2e-04
## Levene     58.900915644   2e-04
```

```
# Evolution des statistiques dans le temps
Gdiff <- seqdiff(biofam.seq,
                 group = biofam$sex,
                 seqdist.args = list(method = "OM", indel = 1, sm = "TRATE"))
title <- "Evolution of the pseudo R2 and L"
plot(Gdiff, stat = c("Pseudo R2", "Levene"),
     lwd = 2, main = title, legend.pos = "topright")
```




```
# Evolution de la divergence au sein des groupes dans le temps
title <- "Evolution of within-group and overall discrepancies"
plot(Gdiff, stat = c("discrepancy"),
     lwd = 2, main = title, legend.pos = "topright")
```



Enfin, il est possible de construire un arbre de divergence (*discrepancy tree*) avec la fonction `seqtree()` qui applique un algorithme récursif permettant d'identifier automatiquement, parmi les covariables, lesquelles discrimines le plus le parcours aboutissant à un arbre de décision. Le package propose également la possibilité de visualiser l'arbre obtenu avec GraphViz et la fonction `seqtreedisplay()`.

```
# Plusieurs variables, arbre de régression
st <- seqtree(biofam.seq ~ sex + birthyr, data = biofam,
    R = 5000, diss = biofam.om, pval = 0.05)
seqtreedisplay(st, type = "d", border = NA, showtree=FALSE,  filename="C:/Users/malnoeg/Documents/Stage_M2
```

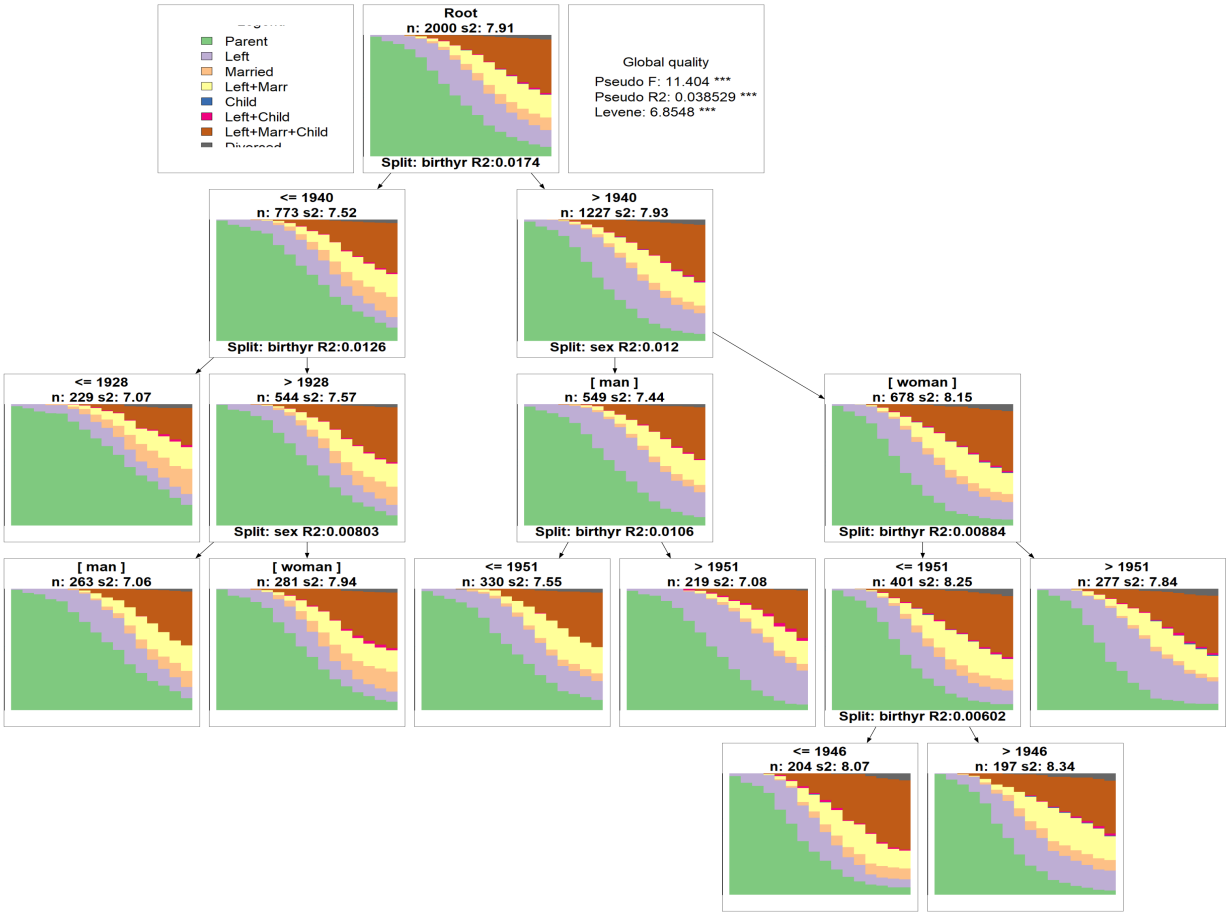


Figure 13: Arbre de divergence

8.2 Analyse entre groupes

La fonction `seqComp()` permet de réaliser un test de rapport de vraisemblance et de calculer la différence du BIC (*Bayesian Information Criterion*) entre deux sous-groupes de séquences. Nous l'illustrons avec la variable `sexe` du jeu de données `biofam`.

```
# Séparation entre homme et femme
indexSex <- biofam["sex"]=="man"
biofam_man.seq <- biofam.seq[indexSex, ]
biofam_woman.seq <- biofam.seq[-indexSex,]
# BIC and Likelihood ratio test for comparing two groups
seqCompare(biofam_man.seq, biofam_woman.seq, method = "OM", indel=1, sm = "TRATE")
```

```
## elapsed time:2.53 secs
```

```
##          LRT  p-value Delta BIC Bayes Factor (Avg) Bayes Factor (From Avg BIC)
## [1,] 2.36935 0.146723 -2.928967          0.2491688                   0.2311974
```

8.3 Analyse de survie

Le package `TraMineR` permet, via la fonction `seqsurv()`, de construire un objet de type `survfit` (issu du package `survival`) à partir de séquences d'états. Cet objet permet ensuite de travailler sur l'évolution de la probabilité de survenue des différents états, et la fonction `seqsplot()` permet de visualiser les fonctions de survie pour l'ensemble des individus ou conditionnellement à une variable catégorielle.

```
seqsurv(biofam.seq)
```

```
## Call: survfit(formula = Surv(spell$dur, spell$status) ~ spell$states,
##     weights = spell$weights, subset = spell$states %in% state)
##
##                       n events median 0.95LCL 0.95UCL
## spell$states=P    1972   1818      8       7       8
```

```
## spell$states=L    896    618      7       6       7
## spell$states=M    246     39     NA      NA      NA
## spell$states=LM   987    611      3       3       3
## spell$states=C      6      3      2       1      NA
## spell$states=LC    40     17     NA       3      NA
## spell$states=LMC  907     24     NA      NA      NA
## spell$states=D     76      0     NA      NA      NA
```

```r
seqsplot(biofam.seq,group=biofam$sex)
```

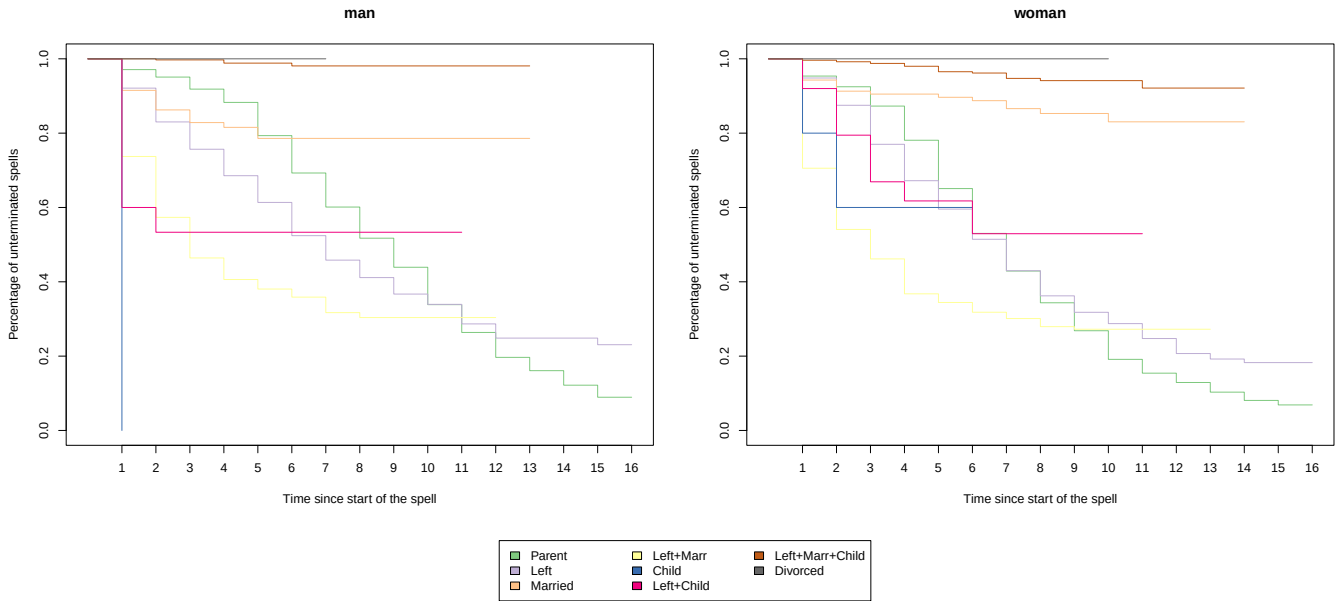


Figure 14: Probabilité de survenue des états selon le sexe

L'objet `survfit` créé est directement compatible avec le package `suvival` qui permet par la suite de réaliser des analyses et de construire des modèles, comme par exemple un modèle de Cox.

8.4 Reformatage pour analyses des transitions et des sous-séquences (CTA, SAMM, SHA)

Les packages `TraMineR` et `TraMineRextras` ne permettent pas de réaliser directement certaines autres analyses classiques, telles que l'analyse des risques concurrents ou les modèles multi-états appliqués aux séquences. Ils proposent en revanche des fonctions de transformation des objets de séquences d'états vers des formats directement compatibles avec d'autres packages, facilitant ainsi la mise en œuvre de ces analyses.

8.4.1 Analyse des risques concurrents (*Competing Trajectory Analysis*)

Le package `TraMineRextras` propose, au travers de la fonction `seqcta()`, de convertir un objet de séquences dans un format adapté à la réalisation d'une analyse des risques concurrents (*Competing Trajectory Analysis*). Les données ainsi obtenues peuvent ensuite être exploitées à l'aide des packages `survival`, `cluster`, `mlogit` et `nnet` pour réaliser l'analyse des risques concurrents.

```r
# Conversion format par rapport à l'état initial "P" = vivre chez ses parents
# en regardant les 5 états suivants
cta <- seqcta(biofam.seq, subseq=5, initial.state="P",
              covar=biofam[, c("sex", "birthyr")])
head(cta,13)
```

```
##        id time event lastobs   T1   T2   T3   T4   T5   sex birthyr
## 11674   1    1 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 11675   1    2 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 11676   1    3 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 11677   1    4 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 11678   1    5 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
```

```
## 11679   1    6 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 116710  1    7 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 116711  1    8 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 116712  1    9 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1943
## 116713  1   10  TRUE    TRUE   LM  LMC  LMC  LMC  LMC   man    1943
## 5144    2    1 FALSE   FALSE <NA> <NA> <NA> <NA> <NA>   man    1935
## 5145    2    2  TRUE    TRUE    L    L    L    L    L   man    1935
## 10134   3    1 FALSE   FALSE <NA> <NA> <NA> <NA> <NA> woman    1946
```

```
# Réalisation d'une analyse des risques concurrents via autres packages
## Typologies des trajectoires après avoir quitté le domicile familial
clusterTraj <- seqdef(cta[cta$event, 5:9])
diss <- seqdist(clusterTraj, method="HAM")
pclust <- pam(diss, diss=TRUE, k=5, cluster.only=TRUE)
pclustname <- paste("Type", pclust)
## Modèle Logit pour expliquer les trajectoires selon
## covariables sexe et année de naissance
cta$traj.event <- "No event"
cta$traj.event[cta$event] <- pclustname
summary(mlogit(traj.event~1|time+sex, data=cta, shape="wide", reflevel="No event"))
```

```
##
## Call:
## mlogit(formula = traj.event ~ 1 | time + sex, data = cta, reflevel = "No event",
##     shape = "wide", method = "nr")
##
## Frequencies of alternatives:choice
##  No event    Type 1    Type 2    Type 3    Type 4    Type 5
## 0.9045058 0.0166129 0.0442213 0.0099797 0.0090833 0.0155970
##
## nr method
## 8 iterations, 0h:0m:2s
## g'(-H)^-1g = 5.76E-05
## successive function values within tolerance limits
##
## Coefficients :
##                      Estimate Std. Error  z-value  Pr(>|z|)
## (Intercept):Type 1 -7.218663   0.226299 -31.8988 < 2.2e-16 ***
## (Intercept):Type 2 -3.654752   0.093559 -39.0635 < 2.2e-16 ***
## (Intercept):Type 3 -7.942212   0.297741 -26.6749 < 2.2e-16 ***
## (Intercept):Type 4 -7.746764   0.298314 -25.9685 < 2.2e-16 ***
## (Intercept):Type 5 -7.456460   0.240027 -31.0650 < 2.2e-16 ***
## time:Type 1         0.408131   0.022571  18.0818 < 2.2e-16 ***
## time:Type 2         0.100315   0.012063   8.3162 < 2.2e-16 ***
## time:Type 3         0.420118   0.029172  14.4014 < 2.2e-16 ***
## time:Type 4         0.390913   0.029629  13.1938 < 2.2e-16 ***
## time:Type 5         0.426810   0.023746  17.9738 < 2.2e-16 ***
## sexwoman:Type 1     0.704025   0.126930   5.5466 2.913e-08 ***
## sexwoman:Type 2     0.166718   0.076051   2.1922   0.02837 *
## sexwoman:Type 3     0.877863   0.165541   5.3030 1.139e-07 ***
## sexwoman:Type 4     0.821414   0.172188   4.7704 1.838e-06 ***
## sexwoman:Type 5     0.721490   0.131053   5.5053 3.685e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log-Likelihood: -6895.7
## McFadden R^2:  0.084952
## Likelihood ratio test : chisq = 1280.4 (p.value = < 2.22e-16)
```

8.4.2 Analyse de séquences par modèle multi-états (*Sequence Analysis Multistate Model*)

De la même manière, le package propose également la conversion des données dans un format adapté à la réalisation d'analyse de séquences par des modèles multi-états (*Sequence Analysis Multistate Model*, SAMM) avec les packages `survival`, `cluster`, `coxme` et `nnet`.

```
# Conversion format pour SAMM en regardant les 5 états suivants
## Extraction de toutes les sous-séquences de taille 5
biofam.samm <- seqsamm(biofam.seq, sublength=5, covar=biofam[, c("sex", "birthyr")])
head(biofam.samm, 5)
```

```
##       id time begin spell.time transition s.1 s.2 s.3 s.4 s.5 sex birthyr
## 1167   1    1     1          0      FALSE   P   P   P   P   P man    1943
## 11671  1    2     1          1      FALSE   P   P   P   P   P man    1943
## 11672  1    3     1          2      FALSE   P   P   P   P   P man    1943
## 11673  1    4     1          3      FALSE   P   P   P   P   P man    1943
## 11674  1    5     1          4      FALSE   P   P   P   P   P man    1943
```

```
## Choronogramme à la sortie de chaque état
plot(biofam.samm)
```

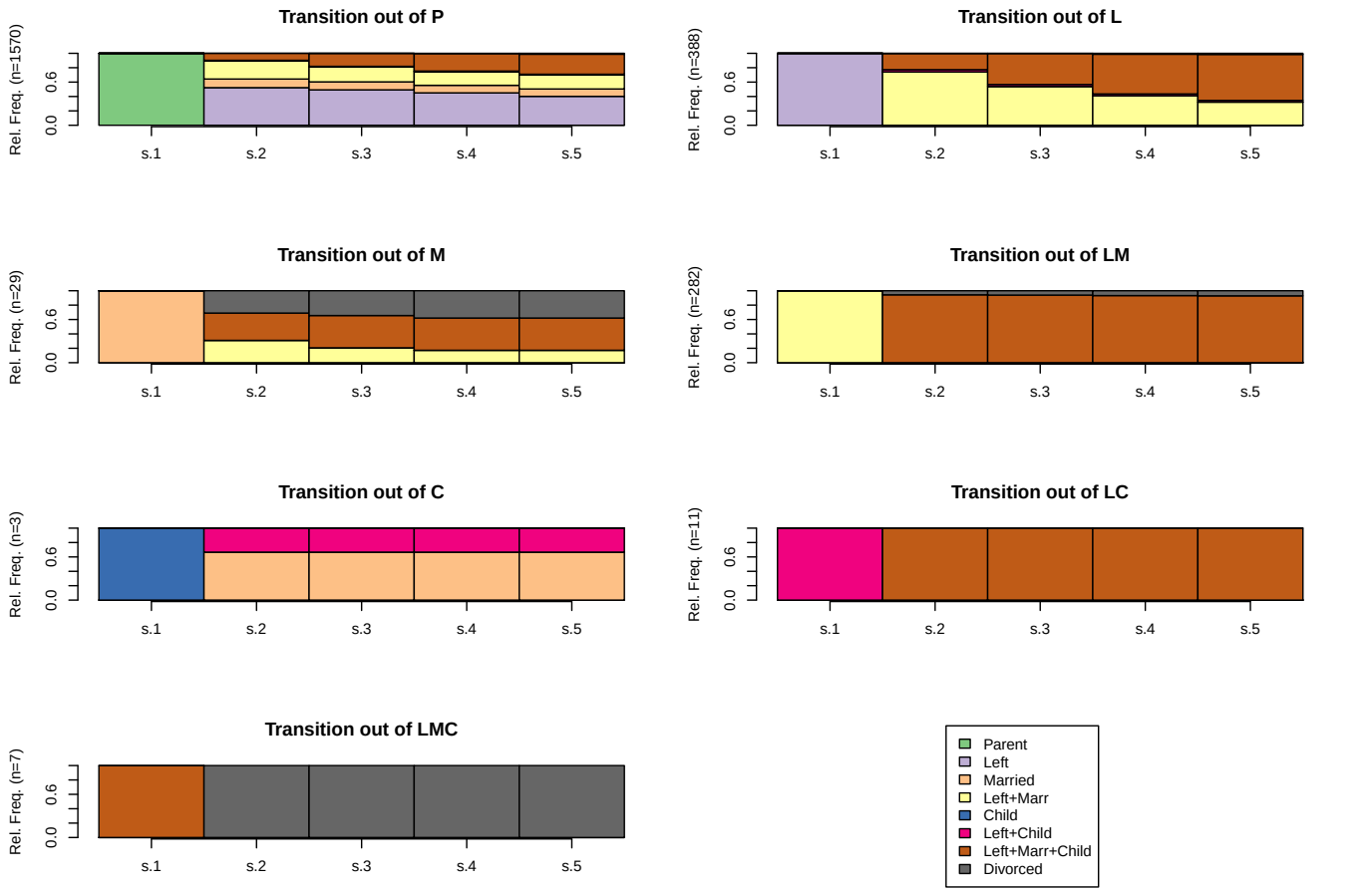


Figure 15: Chronogramme sous-séquences de longueur 5 à la sortie des états de biofam

```
## Objet séquence pour les sous-séquences en sortie de l'état "L"
Pseq <- seqsammseq(biofam.samm, "L")
head(Pseq, 5)
```

```
##       Sequence
## 51410 L-LM-LMC-LMC-LMC
## 4716  L-LM-LM-LM-LMC
## 18468 L-LMC-LMC-LMC-LMC
## 8678  L-LM-LMC-LMC-LMC
## 15717 L-LM-LMC-LMC-LMC
```

```
# Réalisation analyse SAMM avec autres librairies
## Typologie des sous-séquences
Pdist <- seqdist(Pseq, method="LCS")
Pclust <- pam(Pdist, diss=TRUE, k=2, cluster.only=TRUE)
```

```r
Ptypologies <- paste0("Clust", Pclust)
## Competing risk models pour les trajectoires à la sortie de l'état "L"
Peha <- seqsammeha(biofam.samm, "L", Ptypologies)
multinom(SAMMtypology~spell.time+sex+scale(birthyr), data=Peha)
```

8.4.3 Sequence History Analysis

Enfin, la conversion vers un format adapté à l'analyse historique des séquences (*Sequence History Analysis*, SHA) est également proposé via la fonciton `seqsha()`.

```r
# Conversion au format SAH pour l'analyse de la trajectoire précédent
# l'état LMC (Left+Married+Child)
bf.seq2 <- seqrecode(biofam.seq, recodes=list(LMC="LMC"), otherwise = "Other")
dss <- seqdss(bf.seq2)
time <- ifelse(dss[, 1]=="LMC", 1, seqdur(bf.seq2)[, 1])
event <- dss[, 1]=="LMC"|dss[, 2]=="LMC"
sha <- seqsha(biofam.seq, time, event, covar=biofam[, c("sex", "birthyr")])
head(sha, 10)
# Réalisation d'une analyse de SHA avec autres packages
## Objet séquence sur les états précédents LMC
previousTraj <- seqdef(sha[, 4:19])
## Clustering
diss <- seqdist(seqdss(previousTraj), method="LCS")
pclust <- pam(diss, diss=TRUE, k=2, cluster.only=TRUE)
sha$pclustname <- factor(paste("Type", pclust))
## Modèle pour expliquer les typologies
glm(event~time+pclustname+sex, data=sha, family=binomial)
```

9 Autres types de séquences

Enfin, les packages permettent également de définir des séquences d'évènements : une succession ordonnée de transitions, où chaque transition est une liste non-ordonnée d'évènements. La fonction `seqecreate()` permet de créer l'objet séquence d'évènement, la fonction `seqedplot()` permet de visualiser le nombre moyen d'occurence de chaque évènement aux différents points de temps et, de la même façon que pour les séquences d'états, il est possible de calculer la distance entre deux séquences d'évènements par Optimal Matching avec `seqedist()`. Illustrons cela sur nos données factices du début :

$$X = (a, a, a, b, b, c, c) \quad \text{et} \quad Y = (a, a, a, c, c, c, c)$$

que l'on peut voir comme :

$$X = (a) \xrightarrow{3} (a \text{ vers } b) \xrightarrow{2} (b \text{ vers } c)$$

$$Y = (a) \xrightarrow{3} (a \text{ vers } c)$$

```r
# Données
donnees_tse
```

```
##   id time event
## 1  1    0     a
## 2  1    3   a>b
## 3  1    5   b>c
## 4  2    0     a
## 5  2    3   a>c
```

```r
# Création objet séquence d'évènements
data.eseq <- seqecreate(id=donnees_tse$id, timestamp=donnees_tse$time,
    event=donnees_tse$event)
data.eseq[1]
```

```
## [1] (a)-3-(a>b)-2-(b>c)
```

```
data.eseq[2]
```

```
## [1] (a)-3-(a>c)
```

```
# Plot
seqedplot(data.eseq, type="hazard", main="", with.legend = "right" )
```

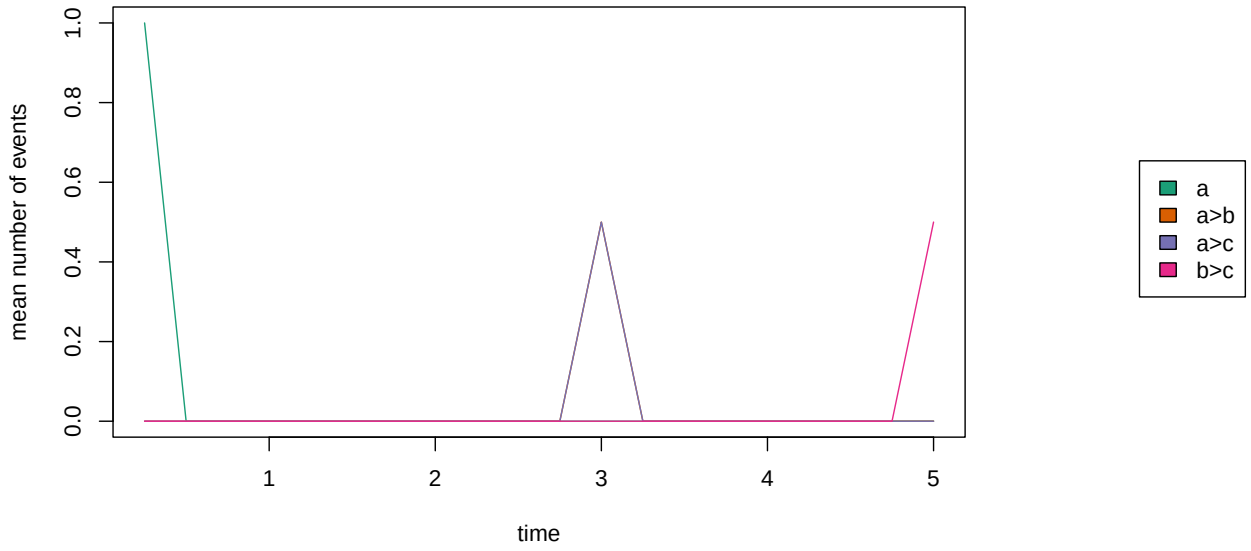


Figure 16: Nombre moyen de transitions à chaque pas de temps

```
# Calcul de distance avec Optimal Matching spécialement pour les séquences
# d'évènements
seqedist(data.eseq, idcost = rep(1,4), vparam=0.1)
```

```
##      [,1] [,2]
## [1,] 0.00 0.75
## [2,] 0.75 0.00
```